

HEAP-URI

1. **Min-Heap / Max-Heap** = un arbore binar **complet** cu proprietatea că valoarea din orice nod este mai mică / mai mare decât valorile celor 2 fii / descendenți direcți (pentru valorile celor 2 fii nu există nicio restricție).

2. **Observație:**

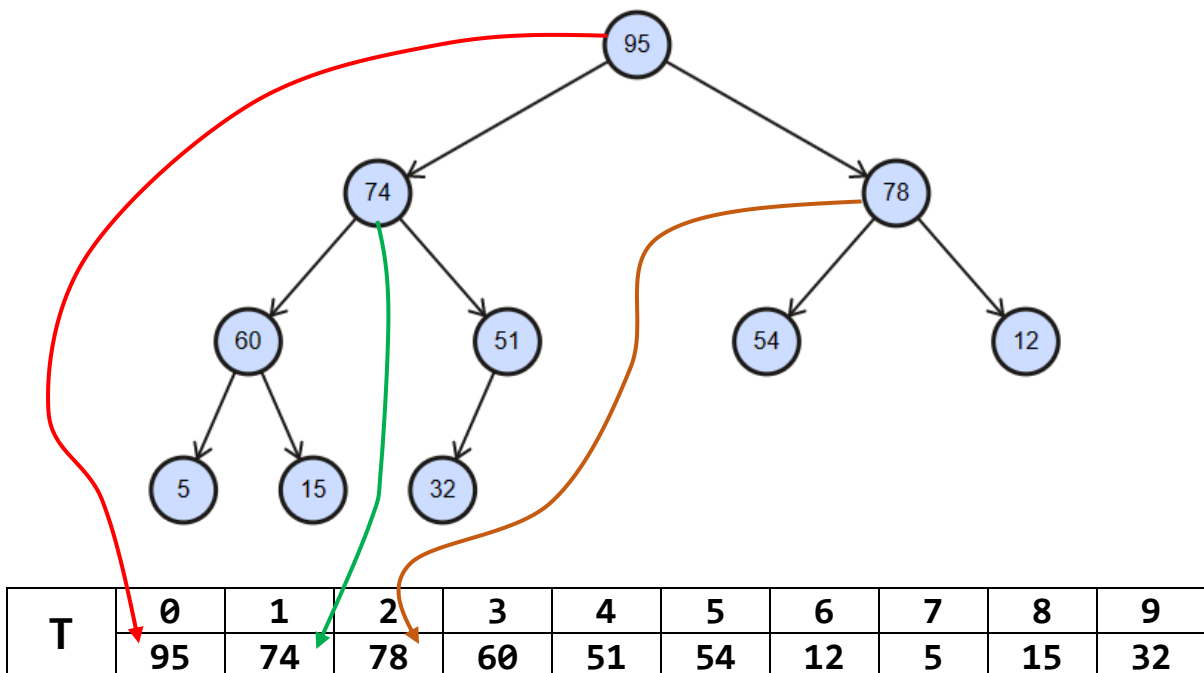
Într-un Min-Heap / Max-Heap valoarea din rădăcină este minimul / maximul tuturor valorilor nodurilor!

3. **Vizualizări Max-Heap-uri:**

- <https://heapsortvisualizer.web.app/>
- <https://www.cs.usfca.edu/~galles/visualization/Heap.html>
- <https://web.archive.org/web/20240608221807/http://btv.melezinek.cz/binary-heap.html>

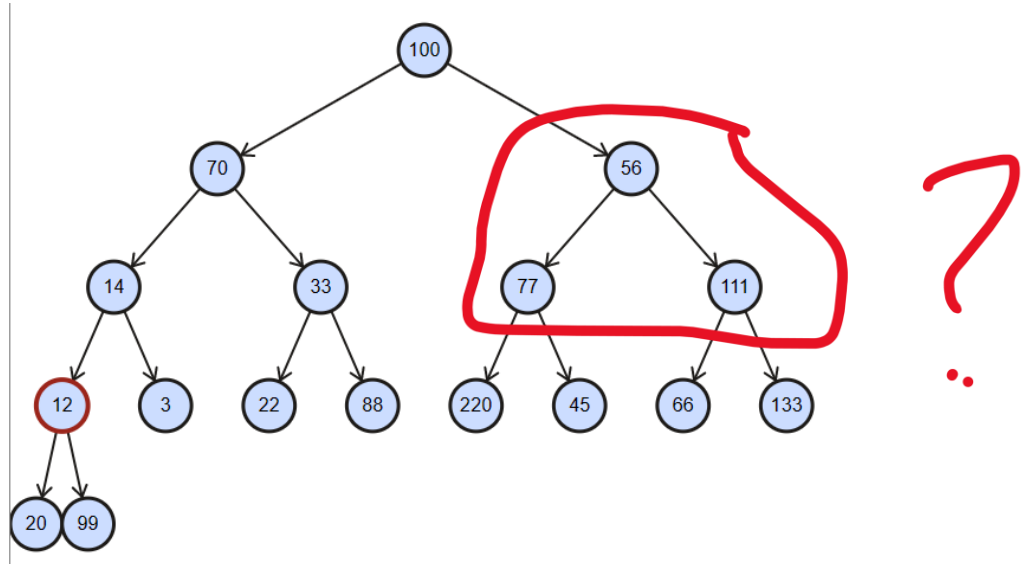
4. **Memorarea unui heap într-un tablou unidimensional / listă:**

- $T[0]$ = rădăcina heap-ului
- $T[2 * i + 1]$ = fiul stâng al elementului $T[i]$
- $T[2 * i + 2]$ = fiul drept al elementului $T[i]$
- $T[(i-1) // 2]$ = părintele elementului $T[i]$

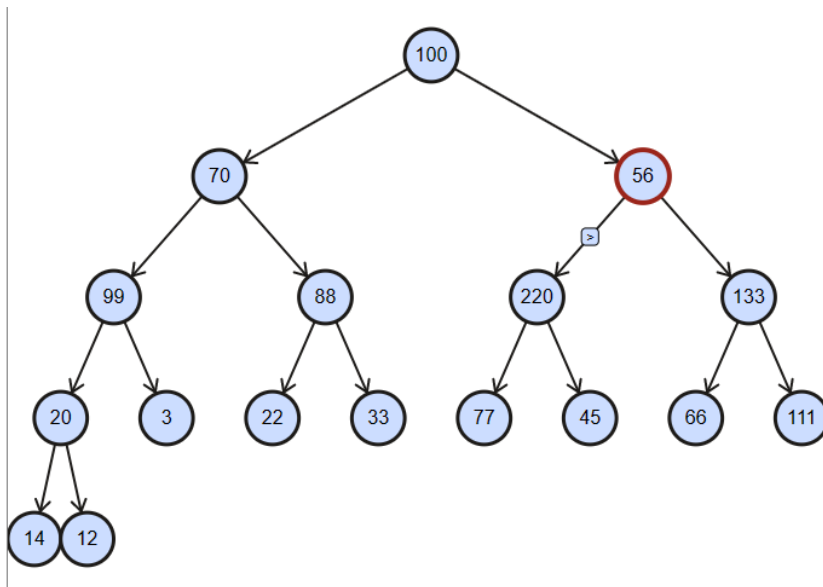


$T = [100, 70, 56, 14, 33, 77, 111, 12, 3, 22, 88, 220, 45, 66, 133, 20, 99]$

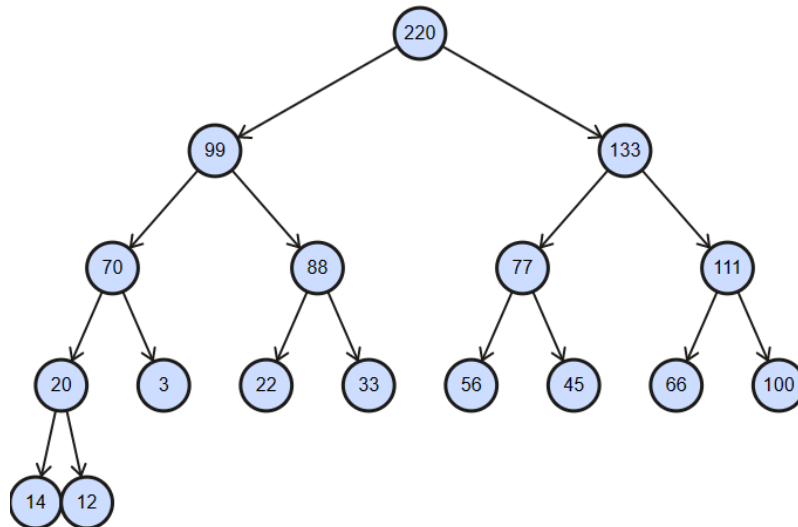
- Arborele binar inițial (care va fi transformat într-un Max-Heap printr-o parcurgere pe niveluri, de jos în sus):



- Arborele binar după ce subarborii cu rădăcinile pe antepenultimul nivel au fost transformați în Max-Heap-uri:



- Arborele binar transformat într-un Max-Heap:



$T = [220, 99, 133, 70, 88, 77, 111, 20, 3, 22, 33, 56, 45, 66, 100, 14, 12]$

5. Implementare Max-Heap în Python

6. Complexitatea medie a operațiilor specifice unui Min-Heap / Max-Heap cu n elemente:

- consultarea valorii minime / maxime: $\mathcal{O}(1)$
- extragerea valorii minime / maxime: $\mathcal{O}(\log_2 n)$
- inserarea unei valori: $\mathcal{O}(\log_2 n)$
- crearea unui Min-Heap / Max-Heap: $\mathcal{O}(n)$

7. Heapsort – complexitate medie $\mathcal{O}(n \log_2 n)$

8. Modulul `heapq` conține o implementare procedurală a unui Min-Heap

(<https://docs.python.org/3/library/heapq.html>)

9. Coadă cu priorități (priority queue) = o structură de date de tip coadă (queue) în care operațiile de inserare și extragere se efectuează în raport de prioritățile atașate elementelor, ci nu de ordinea dată de inserarea lor în coadă.

10. Clasa PriorityQueue din modulul queue reprezintă o implementare obiectuală a unei cozi cu priorități de tip Min-Heap (<https://docs.python.org/3/library/queue.html>)

11. Aplicații:

- Determinarea celor mai mici / mai mari k valori dintr-o listă (<https://www.geeksforgeeks.org/kth-smallest-largest-element-in-unsorted-array/>).
- Se consideră n țevi pentru care se cunosc lungimile lor. Știind că la un moment dat pot fi sudate doar două țevi, iar sudarea a două țevi cu lungimile p și q are un cost egal cu $p + q$, să se determine o modalitate de sudare a tuturor celor n țevi astfel încât costul total să fie minim (<https://www.geeksforgeeks.org/optimal-file-merge-patterns/>).

Exemplu:

Variantă 1 – țevile se sudează în ordinea dată

$L = [50, 20, 100, 60, 40, 20, 70, 10, 80, 100, 90]$	cost total = 0
$L = [50+20, 100, 60, 40, 20, 70, 10, 80, 100, 90]$	cost total = $0 + 70 = 70$
$L = [70, 100, 60, 40, 20, 70, 10, 80, 100, 90]$	
$L = [70+100, 60, 40, 20, 70, 10, 80, 100, 90]$	cost total = $70 + 170 = 240$
$L = [170, 60, 40, 20, 70, 10, 80, 100, 90]$	
$L = [170+60, 40, 20, 70, 10, 80, 100, 90]$	
$L = [230, 40, 20, 70, 10, 80, 100, 90]$	cost total = $240 + 230 = 470$
$L = [270, 20, 70, 10, 80, 100, 90]$	cost total = $470 + 270 = 740$
$L = [290, 70, 10, 80, 100, 90]$	cost total = $740 + 290 = 1030$
$L = [360, 10, 80, 100, 90]$	cost total = $1030 + 360 = 1390$
$L = [370, 80, 100, 90]$	cost total = $1390 + 370 = 1760$
$L = [450, 100, 90]$	cost total = $1760 + 450 = 2210$
$L = [550, 90]$	cost total = $2210 + 550 = 2760$
$L = [640]$	cost total = $2760 + 640 = 3400$

Variantă 2 (optimă) – la fiecare pas se sudează cele mai scurte țevi

$L = [50, 20, 100, 60, 40, 20, 70, 10, 80, 100, 90]$	cost total = 30
$L = [50, 30, 100, 60, 40, 20, 70, 80, 100, 90]$	cost total = $30 + 50 = 80$
$L = [50, 50, 100, 60, 40, 70, 80, 100, 90]$	cost total = $80 + 90 = 170$
$L = [50, 90, 100, 60, 70, 80, 100, 90]$	cost total = $170 + 110 = 280$
$L = [110, 90, 100, 70, 80, 100, 90]$	cost total = $280 + 150 = 430$
$L = [110, 90, 100, 150, 100, 90]$	cost total = $430 + 180 = 610$

L = [110, 180, 100, 150, 100]

L = [110, 180, 200, 150]

L = [260, 180, 200]

L = [260, 380]

cost total = 610 + 200 = 810

cost total = 810 + 260 = 1070

cost total = 1070 + 380 = 1450

cost total = 1450 + 640 = 2090